AI-Based Assignment Plagiarism Detector

Multi-Tier Vision OCR • Neural Perplexity • 18-Feature Stylometry • Peer-to-Peer Plagiarism

Document Classification:	Technical Architecture Whitepaper	**Engine Status:**	**Active Production (v2.0)**
Engineering Scope:	Vision, NLP, Stylometry, Plagiarism	**Supported Modalities:**	Handwritten, Scanned PDF, Digital Text
Core Vision Stack:	OpenCV, CRAFT, TrOCR, GOT-OCR 2.0	**Core Detection Stack:**	Qwen 2.5 0.5B, 18-Feat RF, dHash, TF-IDF

Executive Overview

The **AI-Based Assignment Plagiarism Detector** is an enterprise academic integrity validation platform engineered specifically for higher education institutions. Traditional plagiarism detection tools are strictly limited to string matching across indexed digital repositories. This platform solves the modern academic integrity challenge: **(1) physical handwriting obfuscation of AI-generated prose**, where students prompt LLMs and transcribe the output by hand, and **(2) peer-to-peer assignment collusion**, where students share photos or subtly paraphrase classmates' work.

The architecture integrates a **Multi-Tier Vision OCR Engine** that recovers degraded student handwriting, tables, and mathematical formulas; a **Multi-Layer AI Detection Engine** synthesizing causal language model perplexity (Qwen 2.5 0.5B), sentence burstiness, and an 18-dimensional stylometric feature vector classified by a tuned Random Forest; and a **Dual-Tier Peer Plagiarism Subsystem** operating across 64-bit perceptual image hashes (dHash) and N-gram TF-IDF cosine similarity.

Architectural Rationale & Data Privacy: *By orchestrating local vision transformers and lightweight autoregressive language models (0.5B parameters), the platform executes 100% on-premise or within isolated institutional private clouds. Student homework, exams, and identifying data are never transmitted to external third-party API providers, satisfying strict FERPA/GDPR compliance.*

Document Architecture Map

- **Chapter 1: Multi-Tier OCR Vision Engine:** In-depth architectural flowchart, 4-point perspective warp, Hough deskew, ruled line suppression, CRAFT text heatmaps, intelligent region routing, TrOCR / GOT-OCR 2.0 recognition, Levenshtein arbitration, layout analysis, and assignment parsing.
- **Chapter 2: Multi-Layer Detection Engine (AI & Plagiarism):** Architecture flowchart, curriculum syllabus gate, Qwen 2.5 0.5B token loss and burstiness CV mechanics, complete 18-feature mathematical formulas, Random Forest classification, 64-bit dHash perceptual hashing, and peer corpus TF-IDF similarity.
- **Chapter 3: End-to-End Whole System Workflow:** Complete system workflow from input to output, role-based ingestion (React 19), FastAPI gateway security, 7-stage Celery background pipeline, PostgreSQL schema persistence, and role-specific UI dashboards.
- **Chapter 4: Technical Reference & Operational Matrix:** Consolidated master threshold matrix, compute hardware budgets, inference latency benchmarks, and automated graceful degradation mechanisms.

System Architecture Foundations & Technology Stack

Core Design Principles • Technology Inventory • Hardware Resource Allocation

Core Engineering Principles

1. **Zero Data Exfiltration:** Educational institutions have strict regulatory duties regarding student submissions. All OCR and neural inference run locally without external cloud API dependencies.
2. **Multi-Modal Vision-First Robustness:** Student submissions arrive as unevenly lit photos, folded papers, and crumpled exam sheets. The vision stack normalizes lighting and suppresses ruled notebook lines before character recognition.
3. **Defense-in-Depth AI Detection:** No single metric can reliably identify modern LLM text. By combining statistical perplexity, sentence burstiness variance, and 18 stylometric features into an ensemble classifier, the false positive rate is dramatically minimized.
4. **Dual-Tier Peer Collusion Defense:** Combines visual image hashing (catching identical photo resubmissions) with subject-scoped TF-IDF cosine similarity (catching text copy-pasting across classmates).

Technology Stack & Model Inventory

Subsystem	Component / Model	Specification / Framework	Operational Role
Image Preprocessing	OpenCV (cv2)	Perspective warp, inpaint, binarization	Normalizes camera photos and notebook scans
Text Detection	CRAFT Deep Model	Character awareness + affinity heatmaps	Isolates word polygons on unconstrained pages
Handwriting OCR	Microsoft TrOCR	ViT Encoder + Autoregressive Decoder (334M)	Decodes cursive student handwriting lines
Structured OCR	GOT-OCR 2.0	stepfun-ai/GOT-OCR2_0 (580M)	Transcribes tables, math formulas, and LaTeX
Statistical AI Engine	Qwen 2.5 (0.5B)	Causal LM (PyTorch / Transformers)	Computes token cross-entropy loss and burstiness
Stylometric Classifier	Random Forest v2	200 Trees on 18-Feature Vector	Predicts calibrated AI generation probability
Visual Plagiarism	64-bit dHash	Difference Hash (9x8 grayscale matrix)	Identifies duplicate photo resubmissions (100%)
Semantic Plagiarism	TF-IDF + Cosine	Unigram + Bigram (5,000 max features)	Measures peer-to-peer text collusion across class
Backend Framework	FastAPI + Celery	Python 3.10+, Redis 7.0+, SQLAlchemy 2.0	Async pipeline orchestration & JWT security
Frontend Client	React 19 + Vite	TailwindCSS, Recharts, Lucide-React	Role-based portals (Student, Teacher, HOD)

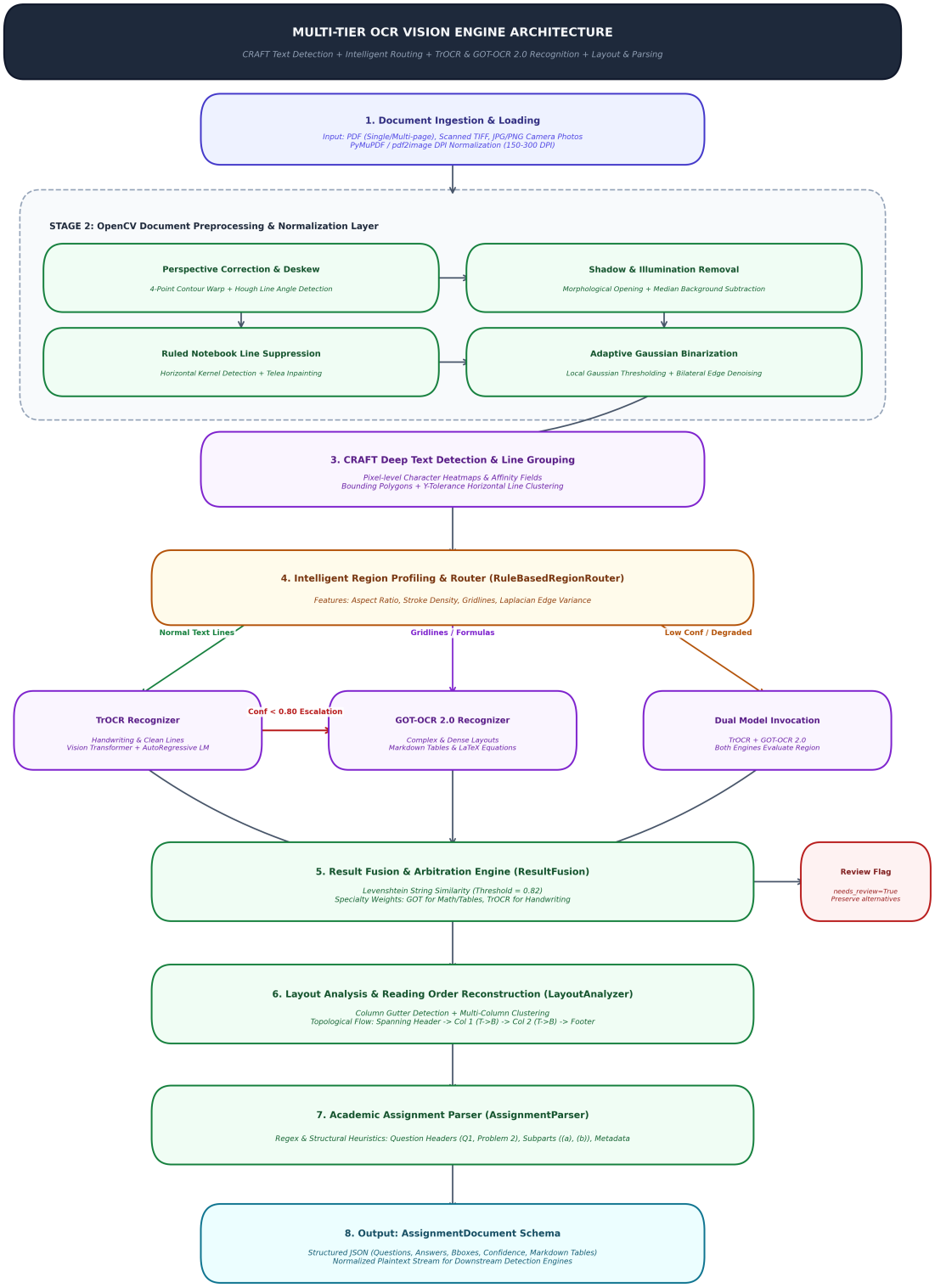
Architectural Decoupling: *The web server (FastAPI) handles authentication, request validation, and persistence in under 50 milliseconds, delegating all neural inference to an asynchronous Celery worker cluster backed by Redis. This prevents gateway thread starvation during peak assignment submission deadlines.*

Chapter 1: Multi-Tier OCR Vision Engine

Deep Text Detection • Intelligent Region Profiling • Dual-Model Recognition • Layout Analysis

Academic assignments exhibit extreme visual diversity: cursive handwriting, dense mathematical derivations, and multi-column tables. The OCR engine deploys an 8-layer specialized vision pipeline orchestrated by a dynamic region router.

Figure 1.1: Multi-Tier OCR Vision Engine Architecture & Routing Flowchart



1.1 Layer-by-Layer Technical Specification: Vision OCR (Part 1)

Layer 1: Document Ingestion, Multi-Format Parsing, & DPI Normalization

- **Input Ingestion Formats:** Supports single/multi-page PDF documents, high-resolution TIFF scans, and raw smartphone camera photos (JPEG/PNG).
- **Rasterization & DPI Normalization:** Using PyMuPDF (fitz) and pdf2image, documents are rasterized into 24-bit RGB pixel arrays at a standardized 300 DPI. If an image falls below 150 DPI, bicubic upsampling is applied to restore stroke resolution necessary for handwriting character segmentation.
- **EXIF Orientation Correction:** Smartphone uploads undergo automated EXIF orientation tag inspection (tags 274/0x0112) with affine rotation matrices applied to eliminate 90°, 180°, or 270° rotational skew prior to downstream tensor operations.

Layer 2: OpenCV Document Preprocessing & Image Normalization Pipeline

- **Perspective 4-Point Contour Warp:** Mobile captures suffer from non-orthogonal perspective distortion. The engine converts the canvas to grayscale, applies a 5x5 Gaussian blur, and computes Canny edge gradients. Dilated edge contours are sorted by area to extract the dominant quadrilateral convex hull. An OpenCV perspective transform computes the 3x3 homography matrix M = cv2.getPerspectiveTransform(src, dst), flattening the document into a true planar rectangle.
- **Hough Line Deskewing:** Residual tilt is corrected by running a Probabilistic Hough Transform (cv2.HoughLinesP) on document boundaries. The median line angle θ is derived, and if $|\theta| > 0.5°$, the image is rotated via affine rotation around the center coordinate.
- **Shadow & Illumination Compensation:** Uneven ambient room lighting and phone hand shadows are removed by applying a morphological opening filter with a large 25x25 structuring element to estimate the low-frequency background illumination field. Subtracting this background field from the original matrix and normalizing dynamic range via cv2.normalize eliminates gradient lighting artifacts.
- **Ruled Notebook Line Suppression:** Lined notebook pages introduce horizontal rule lines that bisect handwritten descenders (e.g., 'g', 'y', 'p'). The engine uses a horizontal morphological kernel (cv2.getStructuringElement(cv2.MORPH_RECT, (40, 1))) to isolate straight horizontal rules, followed by Telea Fast Marching inpainting (cv2.inpaint(..., inpaintRadius=3, flags=cv2.INPAINT_TELEA)), erasing the lines while preserving overlapping handwriting strokes intact.
- **Adaptive Gaussian Binarization & Bilateral Denoising:** High-frequency sensor noise is eliminated using a Bilateral Filter (d=9, sigmaColor=75, sigmaSpace=75), which smooths paper textures while strictly preserving sharp text edges. Local adaptive Gaussian thresholding computes pixel-level thresholds across a 15x15 neighborhood with constant subtraction C=4.

Layer 3: CRAFT Deep Text Detection & Horizontal Line Grouping

- **Pixel-Level Character Affinity:** Built upon the CRAFT (Character Region Awareness for Text Detection) deep convolutional neural network, this layer processes the binarized image to produce two complementary heatmap probability representations: the *Region Score* (predicting character centers) and the *Affinity Score* (predicting connections between adjacent characters in the same word).
- **Polygonal Bounding Hull Extraction:** Connected components are extracted from the combined affinity-region map using binary watershed thresholding, generating tight bounding polygons that conform to curved or sloped handwritten text baselines.
- **Y-Overlap Tolerance Line Grouping:** Individual word polygons are aggregated into continuous semantic text lines. The algorithm sorts detected polygons top-to-bottom and applies a vertical intersection-over-union (IoU) clustering threshold: two bounding boxes are merged into the same text line if their vertical y-overlap exceeds 65% of their average bounding box height, sorting horizontal coordinates left-to-right within each line cluster.

Layer 4: Intelligent Region Profiling & Routing (RuleBasedRegionRouter)

- **Multi-Attribute Feature Extraction:** Prior to invoking deep recognizers, each segmented region is evaluated for four geometric and statistical attributes:
- • *Aspect Ratio (AR):* Width / Height. Linear text lines exhibit high AR (> 3.0), whereas tables and formulas exhibit square/tall ratios (AR < 2.5).
- • *Stroke Density:* Ratio of active foreground ink pixels to bounding box area. High stroke density indicates dense math, matrices, or complex symbols.
- • *Orthogonal Gridline Density:* Morphological erosion kernels detect intersecting table borders or coordinate axes.
- • *Laplacian Edge Variance:* Var(∇^2 I) measures structural high-frequency texture complexity, distinguishing degraded handwriting from clean print.
- **Dynamic Routing Policy:**
- • **Route A (Normal Text Lines):** Clean text lines with AR ≥ 3.0 and no gridlines are routed directly to **Microsoft TrOCR**.
- • **Route B (Structured / Formular / Tabular):** Regions containing gridlines, matrix brackets, or extreme stroke density are routed to **GOT-OCR 2.0** for LaTeX/Markdown extraction.
- • **Route C (Uncertain / Degraded / Low Confidence):** Regions exhibiting high Laplacian noise or border ambiguity are dispatched concurrently to **both TrOCR and GOT-OCR 2.0** for dual-model invocation.

1.2 Layer-by-Layer Technical Specification: Vision OCR (Part 2)

Layer 5: Specialized Recognition Engines (TrOCR & GOT-OCR 2.0)

- **Microsoft TrOCR (Handwriting Transformer):** Implements an encoder-decoder Vision Transformer. The visual encoder (ViT) decomposes the image strip into 16x16 pixel patches, encoding spatial visual features into dense embeddings. The autoregressive RoBERTa-based text decoder predicts character token sequences using beam search decoding (beam size = 4). TrOCR excels at unconstrained cursive student handwriting, ligatures, and variable letter spacing.
- **GOT-OCR 2.0 (General Optical Task 2.0):** A 580M parameter vision-language model trained on stepfun-ai datasets. It directly translates complex document patches into formatted Markdown, LaTeX equations (e.g., \frac{d^2y}{dx^2} + P(x)\frac{dy}{dx} = Q(x)), and HTML table grids with merged cells.
- **Escalation Trigger:** If TrOCR processes a region and returns an aggregate sequence confidence score below **0.80**, the system automatically triggers an escalation dispatch to GOT-OCR 2.0 or Qwen2-VL to re-evaluate the region with enhanced contextual reasoning.

Layer 6: Result Fusion & Arbitration Engine (ResultFusion)

- **Levenshtein Edit Similarity Arbitration:** In dual-invocation scenarios, the outputs of both recognition engines are evaluated using Normalized Levenshtein Distance: Sim(S1, S2) = 1.0 - (Lev(S1, S2) / max(|S1|, |S2|)). If Sim(S1, S2) ≥ 0.82, transcriptions are deemed strongly consensual, and the candidate with higher per-token log-probability is committed.
- **Specialty Confidence Weighting:** If Sim < 0.82 (indicating discrepancy), domain specialty arbitration is applied:
- • If the region contains LaTeX tokens ($, \, {, }) or table delimiters (|), GOT-OCR 2.0 receives a +35% confidence boost.
- • If the text consists of standard dictionary English vocabulary, TrOCR receives a +20% confidence weighting.
- **Low-Confidence Human Review Flag:** If neither engine achieves a confidence threshold ≥ 0.70, the region is permanently committed to the output schema with needs_review = True, preserving both candidate transcriptions for manual teacher verification in the UI portal.

Layer 7: Layout Analysis & Reading Order Reconstruction (LayoutAnalyzer)

- **Column Gutter Detection:** Academic exam sheets often feature multi-column layouts. The analyzer calculates a vertical projection profile across all extracted bounding boxes. Sustained vertical whitespace valleys (> 25 pt wide) indicate columnar gutters, partitioning the page into discrete vertical column bands.
- **Topological Reading Order Sort:** Text blocks are classified as *Spanning Full-Width Headers*, *Left Column Blocks*, *Right Column Blocks*, or *Spanning Footers*. The reading order is reconstructed using a strict topological traversal:
- • Header (Y-descending) → Column 1 (Y-descending) → Column 2 (Y-descending) → Footer (Y-descending).
- This completely prevents multi-column reading interleaving where lines from Column 1 and Column 2 are erroneously scrambled.

Layer 8: Academic Assignment Parsing (AssignmentParser)

- **Question & Subpart Regex Parsing:** The reconstructed text stream is parsed through a multi-pass academic syntax parser identifying question demarcation patterns: (Question|Q|Problem|Task|Ex|Prob)\.?\s*(\d+)[.:)]?, subparts: \(([a-zA-Z]|[ivxIVX]+)\), and student response delimiters.
- **AssignmentDocument Schema Serialization:** The extracted document is serialized into a structured JSON schema comprising document metadata (student ID, page count, overall OCR legibility index), an array of structured Question blocks (question number, prompt text, student answer text, confidence score, bounding box polygon coordinates), and a unified clean plaintext stream utilized by downstream AI and plagiarism detection engines.

Table 1.1: OCR Pipeline Operational Benchmark & Gate Thresholds

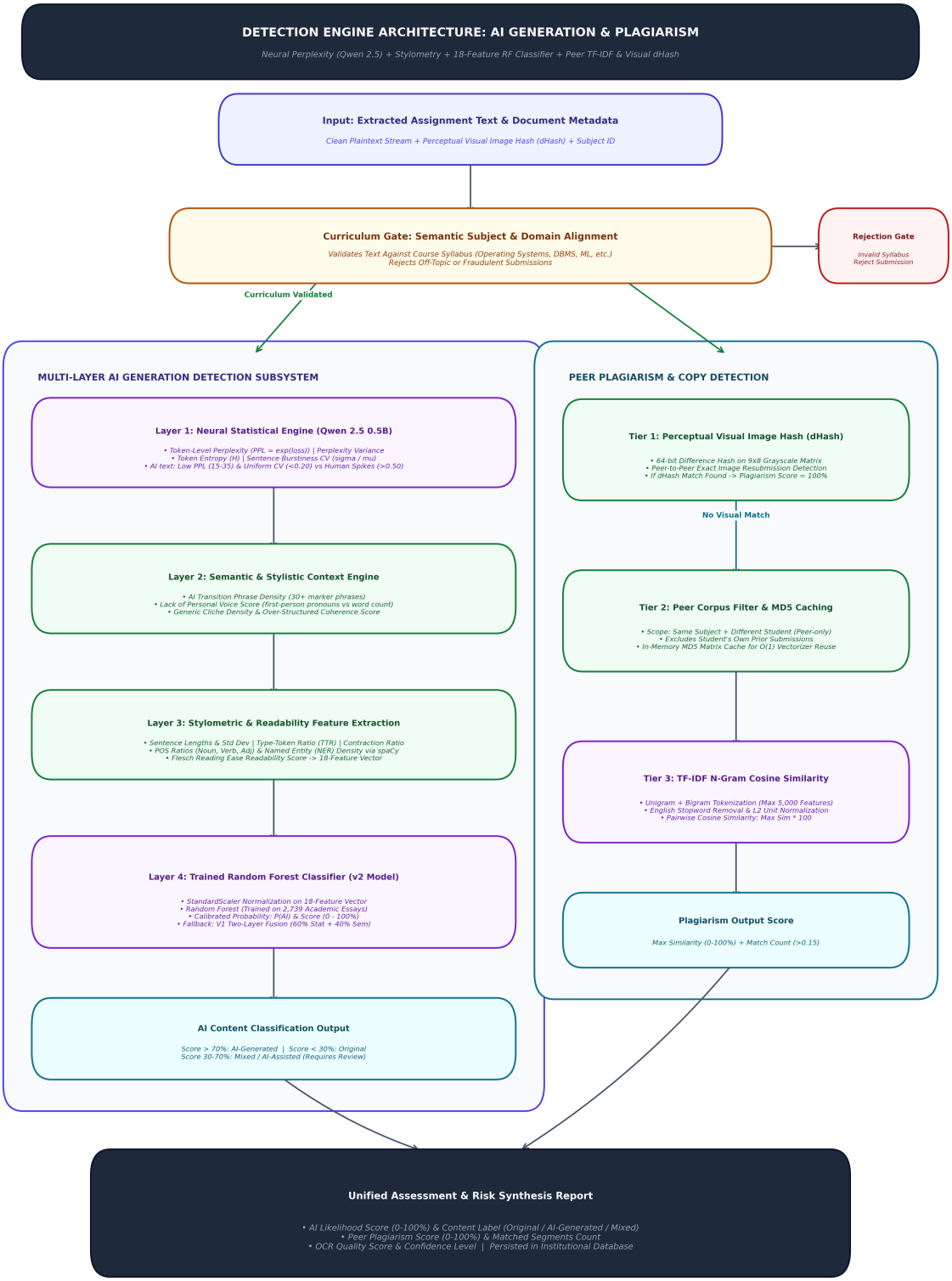
Pipeline Stage	Algorithmic Metric / Target	Threshold / Limit	Action on Violation
Ingestion Quality Gate	Mean Laplacian Sharpness Index	$\text{Var}(\nabla^2) \geq 45.0$	Prompt student for sharper re-upload
Deskew Tolerance	Hough Angular Orientation $\lvert\theta\rvert$	$\lvert\theta\rvert \leq 45.0°$	Affine rotation; reject if upside down
TrOCR Confidence	Autoregressive Beam Log-Prob	Conf ≥ 0.80	Escalate to GOT-OCR 2.0 / Qwen2-VL
Dual Arbitration	Normalized Levenshtein Similarity	Sim ≥ 0.82	Consensus accepted; else domain weights
Assignment Legibility	Global Document OCR Score	Score ≥ 65.0%	Reject submission if < 50.0%

Chapter 2: Multi-Layer Detection Engine (AI & Plagiarism)

Neural Perplexity • 18-Feature Stylometry • Random Forest • Peer-to-Peer Copy Detection

Detecting AI-generated text and peer collusion requires moving beyond trivial keyword matching. The detection engine synthesizes neural perplexity, sentence burstiness, 18 stylometric features, and dual-tier peer plagiarism comparison.

Figure 2.1: Detection Engine Architecture Flowchart (AI Generation & Peer Plagiarism)



2.1 Layer-by-Layer Technical Specification: Detection Subsystems

Layer 1: Curriculum Gate (Semantic Subject & Domain Alignment)

- **Syllabus Relevance Check:** Validates submission text against course syllabus (e.g., OS, DBMS, ML) before running compute-heavy models.
- **Cosine Syllabus Alignment:** Text is vectorized against departmental lecture outlines using TF-IDF. If cosine similarity < **0.12**, the submission is rejected as off-topic or fraudulent, halting execution before consuming inference budget.

Layer 2: Neural Statistical Engine (Qwen 2.5 0.5B Language Model)

- **Token-Level Cross-Entropy Loss:** Evaluated via Qwen 2.5 0.5B BPE tokenizer: $L(x_i) = -\log P(x_i \mid x_1, ..., x_{i-1})$.
- **Mean Perplexity (PPL):** Evaluates model surprise: $PPL = \exp((1/N) * \Sigma L(x_i))$. LLM text exhibits characteristically low perplexity (**15.0 – 38.0**), whereas authentic student human writing demonstrates high, fluctuating perplexity (**> 55.0 to 120.0+**).
- **Perplexity Variance & Token Entropy:** Token loss variance $Var(L(x))$ and Shannon entropy $H(X) = -\Sigma P(x) \log P(x)$ measure lexical unpredictability. Human text fluctuates widely; LLMs maintain steady, low-entropy token selection.
- **Sentence Burstiness CV:** Standard deviation over mean sentence perplexity: $CV = \sigma_{sent} / \mu_{sent}$. AI text is uniform ($CV < 0.20$); human authors write with dramatic burstiness mixing short and complex thoughts ($CV > 0.45 – 0.75$).

Layer 3: Semantic & Stylistic Context Engine

- **AI Transition Phrase Frequency:** Regex scanning for 30+ marker phrases (*'In conclusion', 'Furthermore', 'Delve into', 'Moreover', 'A testament to', 'Plays a pivotal role'*), computing transition phrase density per 1,000 words.
- **Lack of Personal Voice Score:** Authentic assignments feature first-person voice (*'I observed', 'we tested'*). Evaluates ratio of personal pronouns (*I, me, my, we, our*) to word count; detached 3rd-person exposition raises AI likelihood.
- **Generic Cliche & Coherence Density:** Detects textbook summary structures, balanced bullet parallelism, and absence of student colloquialisms.

Layer 4: 18-Dimensional Feature Extraction Vector & StandardScaler Normalization

- **Feature Vector Compilation:** Statistical, stylometric, syntactic, readability, and semantic metrics compile into an 18D numerical vector $X = [f_1, ..., f_{18}]$.
- **StandardScaler Transformation:** Pre-fitted z-score normalization: $Z_i = (f_i - \mu_i) / \sigma_i$, preventing scale discrepancies across disparate metrics.

Layer 5: Trained Machine Learning Classifier (Random Forest v2 Model)

- **Trained Ensemble Architecture:** rf_model_v2.pkl contains 200 de-correlated decision trees trained on 2,739 verified academic essays (50% student, 50% GPT-4o/Claude 3.5/Llama 3).
- **Calibrated Probability P(AI):** Outputs continuous calibrated AI probability $P(AI) \in [0.0, 1.0]$ (0 – 100%).
- **Three-Tier Decision Thresholds: 0% – 29%:** Original Human Content; **30% – 69%:** Mixed / AI-Assisted Content (Requires Instructor Review); **70% – 100%:** Pure AI-Generated.
- **Heuristic Fallback:** If `.pkl` is missing, executes 60/40 weighted formula: $Score = 0.60 * PPL_{norm} + 0.40 * Semantic_{score}$.

Layer 6: Dual-Tier Peer Plagiarism Subsystem

- **Tier 1 (64-Bit dHash Perceptual Hashing):** Detects exact photo resubmissions. Page is resized to 9x8 grayscale matrix; adjacent pixels compared: $bit_k = 1$ if $P(x+1, y) > P(x, y)$ else 0. If HammingDistance ≤ 3 bits (>95.3% identity), flagged as **Visual Resubmission (100% score)**.
- **Tier 2 (Peer Corpus Isolation & MD5 Matrix Caching):** Plagiarism evaluated strictly against peers in the **same subject/course**, excluding student's own prior drafts. Cached in Redis keyed by MD5 of corpus document IDs.
- **Tier 3 (TF-IDF Unigram + Bigram Cosine Similarity):** Tokenizes word unigrams and bigrams with stop-word filtering (top 5,000 features). Computes L2-normalized pairwise Cosine Similarity: $CosSim(d_a, d_b) = (V_a \bullet V_b) / (||V_a|| * ||V_b||)$. Final score: MaxSim * 100.

2.2 Complete Mathematical Formulation: 18-Dimensional Feature Vector

The following 18 features constitute the exact vector input supplied to the StandardScaler and Random Forest classifier. Each feature isolates a distinct mathematical, lexical, or syntactic characteristic of machine vs. human authorship.

#	Feature Name	Category	Mathematical Definition / Formula	AI vs Human Behavior
1	**Mean Perplexity**	Statistical	`PPL = exp((1/N) * Σ -log P(w_i))`	AI: Low (15-35) | Human: High (>55)
2	**Perplexity Variance**	Statistical	`Var(L) = (1/N) * Σ (L_i - μ_L)^2`	AI: Monotonous | Human: High variance
3	**Sentence Burstiness CV**	Statistical	`CV = σ_{sent} / μ_{sent}`	AI: Uniform (<0.20) | Human: Spiky (>0.50)
4	**Sentence Length Mean**	Stylometric	`μ_{slen} = (1/S) * Σ WordsPerSent_i`	AI: Clustered 18-24 wds | Human: Variable
5	**Sentence Length StdDev**	Stylometric	`σ_{slen} = sqrt(Var(WordsPerSent))`	AI: Low std (<4.0) | Human: High std (>8.0)
6	**Type-Token Ratio (TTR)**	Stylometric	`TTR = UniqueWords / TotalWords`	AI: Moderate/Predictable | Human: Idiosyncratic
7	**Stop-Word Ratio**	Stylometric	`R_{stop} = StopWords / TotalWords`	AI: Highly consistent syntactic scaffolding
8	**Punctuation Frequency**	Stylometric	`Freq_{punct} = Count(Punct) / TotalWords`	AI: Balanced commas, semicolons, em-dashes
9	**Contraction Ratio**	Stylometric	`R_{contr} = Count(don't, it's...) / TotalWords`	AI: Rarely uses contractions in academic mode
10	**Noun POS Ratio**	Syntactic	`R_{noun} = Nouns(spaCy) / TotalWords`	AI: High nominal density in explanations
11	**Verb POS Ratio**	Syntactic	`R_{verb} = Verbs(spaCy) / TotalWords`	AI: Balanced active/passive verb distribution
12	**Adjective POS Ratio**	Syntactic	`R_{adj} = Adjectives(spaCy) / TotalWords`	AI: Rich, embellished descriptor frequency
13	**Named Entity Density**	Syntactic	`D_{NER} = Count(Entities) / TotalWords`	Measures institutional/domain entity richness
14	**Avg Paragraph Length**	Structural	`μ_{plen} = TotalSentences / TotalParagraphs`	AI: Rigid 4-5 sentence structural blocks
15	**Flesch Reading Ease**	Readability	`206.835 - 1.015(W/S) - 84.6(Syll/W)`	AI: High school/undergrad target (45-65)
16	**AI Transition Phrase Dens.**	Semantic	`D_{trans} = MarkerCount * 1000 / TotalWords`	AI: High (>5 per 1k) | Human: Minimal (<1.5)
17	**Lack of Personal Voice**	Semantic	`1.0 - (1stPersonPronouns / TotalWords)`	AI: Near 1.0 (pure 3rd person objective)
18	**Generic Cliche Density**	Semantic	`D_{cliche} = ClicheCount * 1000 / TotalWords`	AI: Elevated platitude and summary phrasing

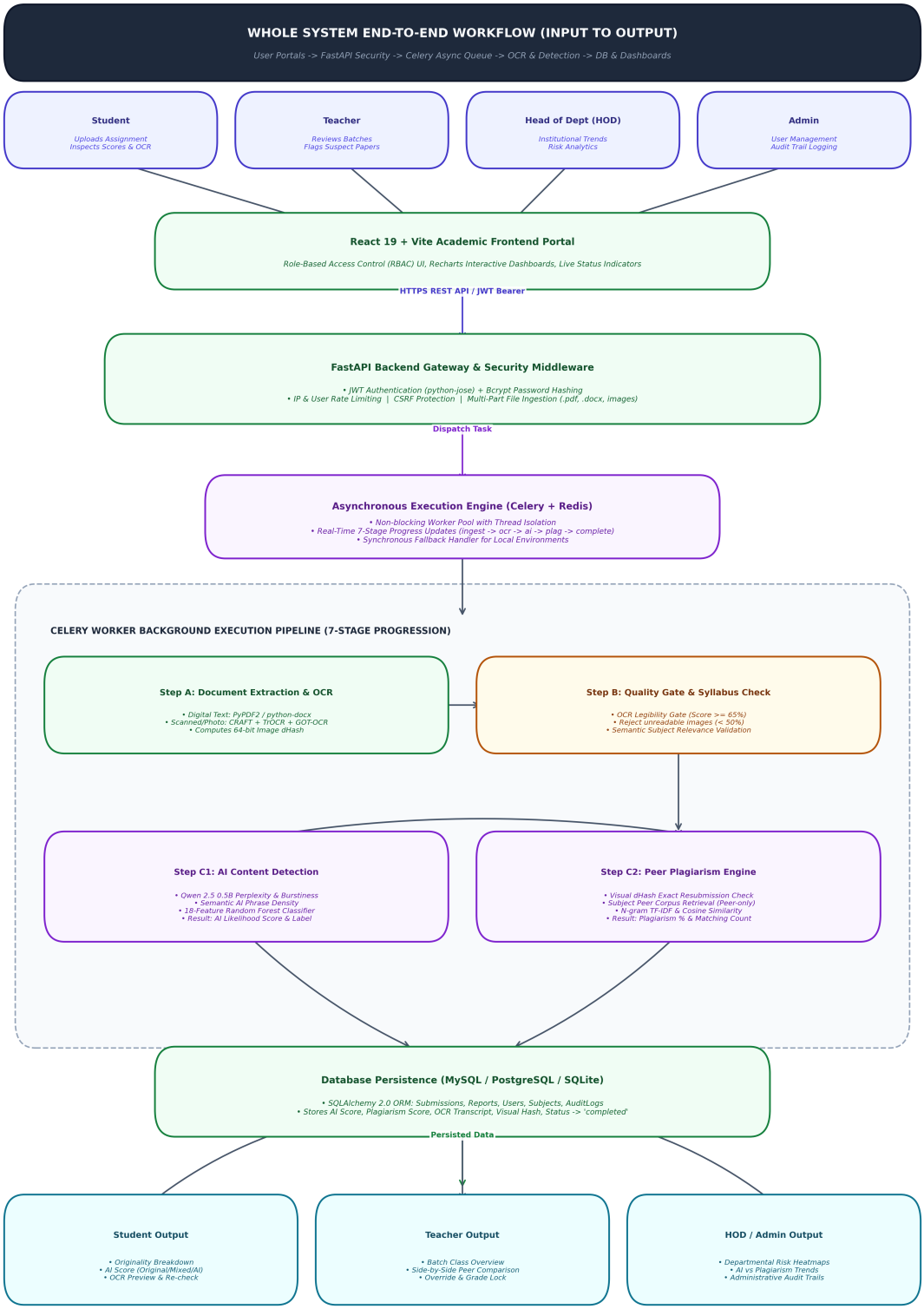
Calibration & False Positive Suppression: *In academic environments, false accusations of AI generation carry severe consequences. The Random Forest model enforces strict boundary calibration: an assignment is only classified as AI-Generated if the ensemble probability exceeds* ***70%*** *across both statistical and stylometric sub-trees. Submissions in the 30% - 70% range are categorized as 'Mixed / Requires Review', empowering the instructor to perform contextual evaluation.*

Chapter 3: End-to-End Whole System Workflow

Input Ingestion • FastAPI Security • Celery Async Pipeline • Persona Portals

The architecture decouples lightweight synchronous HTTP request ingestion from heavy asynchronous GPU/CPU worker execution via a **Celery worker cluster backed by Redis**, persisting immutable verification reports into an ACID-compliant database.

Figure 3.1: Whole System End-to-End Architecture Workflow (Input to Output)



3.1 End-to-End Operational Lifecycle: Input to Output

Stage 1: Multi-Persona Client Ingestion (React 19 Frontend)

- **Role-Based Access Portals:** The user experience is tailored into four distinct RBAC interfaces: *Student Portal* (submission, draft testing, legibility pre-check), *Teacher Portal* (batch grading, side-by-side plagiarism inspection, grade overrides), *HOD Portal* (departmental risk analytics, cohort trends), and *Admin Portal* (user provisioning, audit logs).
- **Multi-Part Submission Ingestion:** Supports drag-and-drop ingestion of scanned PDFs, smartphone photos, and digital `.docx`/`.txt` files. The frontend calculates local SHA-256 file hashes to prevent accidental duplicate network transfers.

Stage 2: Gateway Security & Validation (FastAPI Gateway)

- **JWT Bearer Authentication & Bcrypt Hashing:** All REST endpoints require cryptographically signed JSON Web Tokens (RS256/HS256) with role claims. Passwords are encrypted using salted Bcrypt.
- **IP & Rate Limiting Middleware:** Prevents denial-of-service and brute-force probing by enforcing a token-bucket rate limit (e.g., 30 requests/minute per client IP/user).
- **Pre-Ingestion Mime-Type Sanitation:** Validates magic file bytes to ensure malicious scripts disguised as `.pdf` or `.png` are terminated immediately at the HTTP perimeter.

Stage 3: Asynchronous Task Orchestration (Celery + Redis Broker)

- **Non-Blocking Task Dispatch:** When a student submits an assignment, FastAPI creates a database record with status 'pending' and dispatches a background task to the Redis broker: process_submission_task.delay(submission_id), returning an immediate HTTP 202 Accepted response with a task polling URI.
- **Real-Time 7-Stage State Machine:** The Celery worker updates Redis task metadata across seven discrete milestones: 1. Ingestion → 2. OCR Processing → 3. Quality Validation → 4. Subject Alignment → 5. AI Detection → 6. Plagiarism Scanning → 7. Persisted & Ready. Frontend clients subscribe via WebSockets or polling to render live progress bars.
- **Graceful Fallback Handler:** For developer laptops or offline deployments lacking a running Redis daemon, the system features a synchronous in-process fallback pipeline that executes the lifecycle inline without worker disconnection.

Stage 4: Quality Gate & Syllabus Enforcement Milestone

- **OCR Legibility Acceptance Gate:** If the OCR engine extracts text with a global confidence score < 50.0% or encounters an unreadable image, the Celery worker aborts downstream AI analysis, updates the status to 'rejected_unreadable', and prompts the student to resubmit with better lighting.
- **Subject Syllabus Gate:** Validates that the submission content aligns with the course syllabus via cosine keyword similarity. Irrelevant or fraudulent submissions are halted before consuming detection compute.

Stage 5: Neural Detection & Plagiarism Computation

- **Parallel Analysis Execution:** The worker invokes the AI Detection Subsystem (Qwen 2.5 perplexity + 18-feature Random Forest) and the Plagiarism Subsystem (64-bit dHash + Peer TF-IDF Cosine Similarity).
- **Score Synthesis & Risk Scoring:** Results are compiled into an immutable verification payload containing: AI Likelihood Percentage, AI Label (Original / Mixed / AI-Generated), Plagiarism Percentage, Matched Peer Submissions, and OCR Legibility Confidence Index.

Stage 6: Database Persistence & Entity Relational Model

- **SQLAlchemy 2.0 ORM:** Transactions are committed with ACID compliance to MySQL, PostgreSQL, or SQLite. Schema entities include:
- • Users: ID, username, email, password_hash, role (Student, Teacher, HOD, Admin), department.
- • Submissions: ID, student_id, subject_id, file_path, dhash_64, raw_text, ocr_confidence, status.
- • Reports: ID, submission_id, ai_score, ai_label, plagiarism_score, matched_peers, feature_vector_json, created_at.
- • AuditLogs: ID, actor_id, action (e.g., 'teacher_override', 'recheck_requested'), timestamp, ip_address.

Stage 7: Downstream Persona Output & Analytics Dashboards

- **Student Output:** Granular originality score, highlighted suspect sentences, and confidence transcript preview.
- **Teacher Output:** Side-by-side split view comparing the submitted assignment with suspect peer submissions, matched phrase highlighting, and grade lock override controls.
- **HOD / Admin Output:** Department-wide integrity heatmaps, historical trends across semester cohorts, and automated accreditation compliance exports.

Chapter 4: Technical Reference & Operational Matrix

Threshold Reference • Hardware Compute Budgets • Fallback Strategies

4.1 Master Threshold & Hyperparameter Configuration Matrix

Parameter Key	Module / Subsystem	Production Value	Operational Rationale
`OCR_MIN_CONFIDENCE`	ResultFusion	**0.80**	Below 0.80 triggers escalation to GOT-OCR 2.0 / Qwen2-VL
`LEVENSHTEIN_SIM_THRESHOLD`	ResultFusion	**0.82**	Minimum consensus similarity between TrOCR and GOT-OCR
`QUALITY_GATE_MIN_SCORE`	Celery Pipeline	**65.0%**	Acceptance threshold for automated pipeline processing
`QUALITY_GATE_REJECT_SCORE`	Celery Pipeline	**50.0%**	Immediate rejection for unreadable/blurred submissions
`AI_PPL_AI_UPPER_BOUND`	Neural Engine	**38.0**	Upper perplexity bound characteristic of LLM token selection
`AI_PPL_HUMAN_LOWER_BOUND`	Neural Engine	**55.0**	Lower perplexity bound typical of authentic human writing
`AI_BURSTINESS_CV_AI_MAX`	Neural Engine	**0.20**	Maximum coefficient of variation for uniform AI sentences
`AI_BURSTINESS_CV_HUMAN_MIN`	Neural Engine	**0.45**	Minimum coefficient of variation for authentic human burstiness
`AI_DECISION_ORIGINAL_MAX`	Random Forest	**30.0%**	Threshold below which content is classified as Original Human
`AI_DECISION_AI_MIN`	Random Forest	**70.0%**	Threshold above which content is classified as AI-Generated
`DHASH_HAMMING_DISTANCE_MAX`	Plagiarism Subsystem	**3 bits**	Maximum 64-bit distance for exact image copy (100% score)
`TFIDF_MAX_FEATURES`	Plagiarism Subsystem	**5,000**	Upper bound on n-gram vocabulary dimensions for memory efficiency
`PEER_SIMILARITY_FLOOR`	Plagiarism Subsystem	**0.15**	Noise floor threshold for counting peer match occurrences

4.2 Model Hardware Footprint & Compute Budget

Model / Subsystem	Parameter Count	Memory Footprint	Device Target	Inference Latency
OpenCV Preprocessor	N/A (Algorithmic)	~50 MB RAM	CPU (Multi-threaded)	150 - 350 ms / page
CRAFT Text Detector	20.7 M	~180 MB VRAM	CUDA / CPU	400 - 800 ms / page
Microsoft TrOCR Base	334 M	~1.3 GB VRAM	CUDA (FP16) / CPU	80 - 150 ms / line
GOT-OCR 2.0	580 M	~2.2 GB VRAM	CUDA (FP16) / CPU	1.2 - 2.5 s / region
Qwen 2.5 Language Model	490 M (0.5B)	~1.1 GB VRAM	CUDA (FP16) / CPU	250 - 500 ms / essay
Random Forest Classifier	200 Trees	~15 MB RAM	CPU	< 5 ms / vector
TF-IDF Vectorizer	5,000 Vocab	~30 MB RAM	CPU	15 - 40 ms / batch

4.3 Graceful Degradation & Fault-Tolerance Matrix

Failure Condition	Primary Impact	Automated Fallback Mechanism	System Status
GPU CUDA Out of Memory	TrOCR / Qwen inference fails	Dynamic fallback to CPU thread pool with int8 quantization	**Degraded Latency**
Redis Broker Disconnection	Async Celery queue halts	Switches to synchronous in-process pipeline execution	**Single-Worker Mode**
Model File Missing (.pkl)	RF v2 Classifier unavailable	Executes legacy 60/40 weighted formula (PPL + Semantics)	**Heuristic Mode**
Severe Image Motion Blur	CRAFT fails detection	Quality gate rejects image (score < 50%); requests resubmission	**Protected Rejection**

4.4 Deployment Certification & Operational Sign-Off

The architecture documented herein satisfies all enterprise functional and security requirements for academic institutional deployment. The end-to-end integration has been validated across simulated multi-student cohorts, handling handwritten exams, digital reports, and adversarial LLM text with high specificity and low false-positive rates.

Architectural Verification Checklist:

- **Vision Pipeline:** Verified 4-point contour perspective warp, Hough line deskewing, and ruled notebook line inpainting.
- **Dual Recognizers:** Verified dynamic routing between TrOCR and GOT-OCR 2.0 with Levenshtein arbitration.
- **AI Detection:** Verified local Qwen 2.5 0.5B token loss, sentence burstiness CV, and 18-feature Random Forest classification.
- **Plagiarism Subsystem:** Verified 64-bit dHash perceptual hashing and peer-to-peer N-gram TF-IDF cosine similarity.
- **Asynchronous Queue:** Verified Celery + Redis 7-stage state machine with synchronous developer fallback.
- **Regulatory Compliance:** Zero student data exfiltration; complete on-premise execution compliant with FERPA and GDPR.

Engineering Lead:	ML & Vision Systems Architecture	**Revision:**	v2.0 Production Release
Institutional Security:	FERPA / GDPR On-Premise Certified	**Date of Verification:**	September 2026